Compulsory Assignment 3: Semantic Segmentation With U-net

Please fill out the the group name, number, members and optionally the name below.

Group number:

Member 1: Tobias Tessem Hole
Member 2: Oliver Tschudi Madsen
Member 3: Mehdi Jamali

Group name (optional): swag gangsters

Assignment Submission

To complete this assignment answer the questions in this notebook and write the code required to implement the relevant models. However, we reccomend that you start early. This assignment has subsections, each of which build upon the last. This assignment is completed by doing the following.

- Submit notebook as an .ipynb file to canvas.
- Submit notebook as an .pdf file to canvas.

Your submission must correctly implement the full pipeline to segment the images using a U-Net. High accuracy is not required, as the purpose of this assignment is to evaluate the correct implementation of the U-Net architecture and workflow.

NOTE: It is recommended to use Google Colab with GPU enabled for CA3 taska. Keras and TensorFlow (version ≥2) will automatically use the GPU if one is available

- Read tasks thoroughly before answering.
- Make sure to answer all tasks.
- Ensure all code cells are run.
- Ensure all figures are visible in the PDF.
- Provide a brief explanation of how your code works

How to Use Google Colab for CA3

To work on the CA3 assignment in Google Colab, follow these steps:

---

1. Connect Google Colab to Google Drive

Your CA3 files should be stored in Google Drive so Colab can access them.
Run the following code to mount your Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
```

```
import sys

sys.path.append("/content/drive/MyDrive/dat300/CA3")
```

After running this, you will be asked to grant access. Once mounted, your Drive will be available under `/content/drive/MyDrive/`.

---

2. Upload the CA3 Folder

- First, unzip the **CA3** folder on your local computer.
- Then upload the unzipped CA3 folder into your Google Drive (for example inside `MyDrive`).

Example path: /content/drive/MyDrive/dat300/CA3

Change Runtime to GPU

To use GPU in Google Colab:

1. Go to the top menu and click **Runtime** → **Change runtime type**.
2. In the pop-up window, set **Hardware accelerator** to **GPU**.
3. Click **Save**.

Now your notebook will run on a GPU backend.

---

3. Add CA3 to Python's Search Path

To make sure Python finds the `utilities.py` and other scripts in CA3, add the path (the path is just an example):

import sys

sys.path.append("/content/drive/MyDrive/dat300/CA3")

Adjust the path above to match the actual location of your CA3 folder in Google Drive.

---

4. Open the CA3 Notebook

- Upload the CA3 Jupyter notebook (for example, `CA3.ipynb`) to Colab.
- Once opened, you can run the notebook and it will access the files in your CA3 folder.

---

5. Watch the First Lecture

The first lecture explains how to use Google Colab effectively.
Review it if you are new to Colab.

That's it — you are now ready to run your CA3 code on Google Colab.

Introduction

Chest X-ray(CXR) images are important in medical imaging because it helps focus analysis on the lung region, making disease detection (eg: pneumonia, COVID-19) more accurate and reliable. The purpose is to create clear lung masks that isolate the lungs from surrounding anatomy.

Lung segmentation is the process of separating lung regions from the rest of a chest X-ray. In this assignment, you are provided with CXR images and corresponding binary masks, where pixel values of 0 represent non-lung areas and 1 represents lung tissue. Your task is to focuse on semantic segmentation using a U-Net–style architecture. You will segment lung regions from chest X-ray images.

Dataset

The dataset consists of 200 images with corresponding binary masks:

- **Images:** 200 chest X-ray images (grayscale).
- **Masks:** 200 binary masks.
  - `1` = lung
  - `0` = background

Structure

- Images: `CXR_png/`
- Masks: `masks/`

  Mask filenames may use suffixes `_mask`.

Pairing rule: For `X.png`, expect `X_mask.png` (accepted suffixes).
You must programmatically verify pairs and report any missing masks.

image.png

Assignment Tasks

1. Build a file-pairing script to match each image with its mask; print a summary of matched pairs and missing files.

2. Create train/val/test splits (80/10/10) with a fixed random seed.

3. Implement preprocessing and mask-safe augmentations (flips, crop/zoom; nearest-neighbor resize for masks).

4. Implement the classical U-Net structure that you have learned about in the lectures.

5. Implement a U-Net with Transfer Learning (e.g., ResNet50, VGG16, MobileNetV2).

6. Train your models with a composite loss (e.g., 0.5·BCE + 0.5·DiceLoss), track Dice and IoU metrics, and use callbacks (ModelCheckpoint, EarlyStopping).

7. Evaluate on the test set; report metrics and show qualitative results.

Task 0.1 : Data Organization & Pairing

- Build a file-pairing script to match each image with its mask.

- Print a summary of matched pairs and missing files.

- Show the distribution of training, validation and test set

Set DATA_ROOT to the path of the Images folder located in your CA3 assignment folder.

```
import os, glob, random
import numpy as np
import matplotlib.pyplot as plt
import warnings
import matplotlib
import tensorflow as tf
import tensorflow.keras as ks

import importlib
import utilities as ut
importlib.reload(ut)

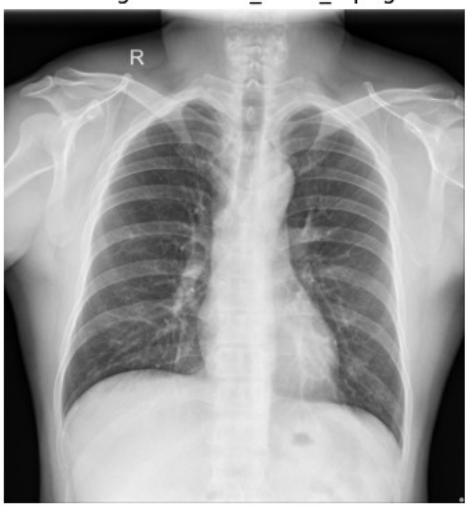
DATA_ROOT = r"/content/drive/MyDrive/dat300/CA3/Images"
IMG_DIR = os.path.join(DATA_ROOT, "CXR_png")
MSK_DIR = os.path.join(DATA_ROOT, "masks")
```

```
pairs, missing = ut.build_pairs(IMG_DIR, MSK_DIR)
ut.report_pairs(pairs, missing)
ut.peek_pairs(pairs, n_show=3)

train, val, test = ut.split_dataset(pairs, seed=42)
ut.plot_split_distribution(train, val, test)
```

```
Total images: 200
Matched pairs: 200
Missing masks: 0
```

Image CHNCXR_0167_0.png



Mask

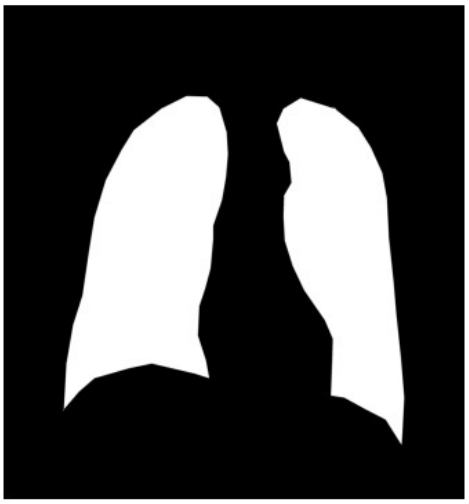


Image CHNCXR_0005_0.png



Mask

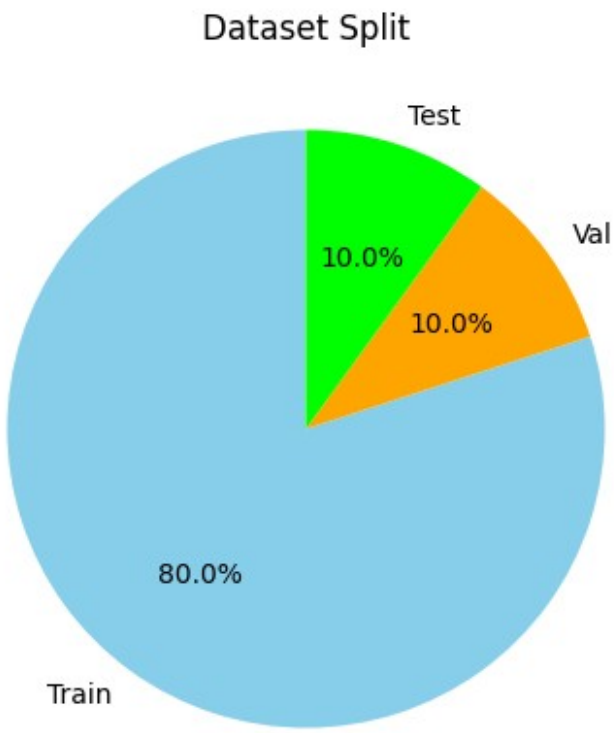


Image CHNCXR_0236_0.png



Mask





Task 0.1 : Preprocessing & Augmentation

- Implement preprocessing for images and masks.
- Ensure masks are not corrupted during transformations.

Image

- Decode as RGB (3 channels), resize to `IMG_SIZE`.
- Use ResNet preprocess.

Mask

- Decode as 1 channel, nearest resize (to preserve labels).
- Binarize.

Augmentations (mask-safe)

- Horizontal flip
- Light zoom/shift via pad + random crop
- Light brightness/contrast jitter (image only)

```
import importlib
import utilities as ut
importlib.reload(ut)
```

```python
# Build datasets
train_ds = ut.make_dataset(train, augment_data=False)   # no
augmentation
val_ds   = ut.make_dataset(val, augment_data=False)
train_aug_ds = ut.make_dataset(train, augment_data=True)
test_ds = ut.make_dataset(test, augment_data=False)

# -------------------------
# Check dataset sizes
# -------------------------
print("Pairs available:")
print("  Train set:", len(train), "pairs")
print("  Val set  :", len(val), "pairs")
print("  Test set :", len(test), "pairs")

# Count actual batches and images
def count_batches_and_images(ds, name):
    n_batches = 0
    n_images = 0
    for imgs, msks in ds:
        n_batches += 1
        n_images += imgs.shape[0]
    print(f"{name}: {n_batches} batches, {n_images} images")

count_batches_and_images(train_ds, "Train dataset (no aug)")
count_batches_and_images(val_ds, "Validation dataset")
count_batches_and_images(train_aug_ds, "Train dataset (with aug)")
count_batches_and_images(test_ds, "Test dataset")

# -------------------------
# Inspect shapes before augmentation
# -------------------------
for imgs, msks in train_ds.take(1):
    print("Before augmentation - images:", imgs.shape, "masks:",
msks.shape)

# Inspect shapes after augmentation
for imgs, msks in train_aug_ds.take(1):
    print("After augmentation - images:", imgs.shape, "masks:",
msks.shape)
```

```
/usr/local/lib/python3.12/dist-packages/tensorflow/python/data/ops/
structured_function.py:258: UserWarning: Even though the
`tf.config.experimental_run_functions_eagerly` option is set, this
option does not apply to tf.data functions. To force eager execution
of tf.data functions, please use
`tf.data.experimental.enable_debug_mode()`.
  warnings.warn(
```

```
Pairs available:
  Train set: 160 pairs
  Val set  : 20 pairs
  Test set : 20 pairs
Train dataset (no aug): 40 batches, 160 images
Validation dataset: 5 batches, 20 images
Train dataset (with aug): 40 batches, 160 images
Test dataset: 5 batches, 20 images
Before augmentation - images: (4, 512, 512, 3) masks: (4, 512, 512, 1)
After augmentation - images: (4, 512, 512, 3) masks: (4, 512, 512, 1)
```

Part 1 - Model Implementation

Intersection over Union

The IoU score is a popular metric in both segmentation and object detection problems.

If you want to use the `plot_training_history` function in the `utilities.py` library remember to compile the model with the TP, TN, FP, FN metrics such that you can estimate the *Intersection-over-Union*.
However, it is voluntary to estimate IoU.(check the `utilities.py`)

See example below:

You have also been provided with a custom F1-score metric in the `utilities.py` library, which is specific for image segmentation.
This is mandatory to use when compiling the model.

Task 1.1: Model Implementation

Implement the classical U-net structure that you have learned about in the lectures.
Feel free to experiment with the number of layers, loss-function, batch-normalization, etc.

Remember to compile with the F1-score metric.

```
#code from lecture notes
#added spatialdropout2d as it drops regions of the feature maps, which
helps when there are large regions of background

from tensorflow.keras.layers import SpatialDropout2D

def conv2d_block(input_tensor, n_filters, kernel_size = 3, batchnorm =
True):
    """Function to add 2 convolutional layers with the parameters
passed to it"""
    # first layer
    x = Conv2D(filters = n_filters, kernel_size = (kernel_size,
```

```
kernel_size),\
              kernel_initializer = 'he_normal', padding = 'same')
(input_tensor)
    if batchnorm:
        x = BatchNormalization()(x)
    x = Activation('relu')(x)

    # second layer
    x = Conv2D(filters = n_filters, kernel_size = (kernel_size,
kernel_size),\
              kernel_initializer = 'he_normal', padding = 'same')(x)
    if batchnorm:
        x = BatchNormalization()(x)
    x = Activation('relu')(x)

    return x


def get_unet(input_img, n_filters = 16, dropout = 0.1, batchnorm =
True, n_classes = 2):
    # Contracting Path
    c1 = conv2d_block(input_img, n_filters * 1, kernel_size = 3,
batchnorm = batchnorm)
    p1 = MaxPooling2D((2, 2))(c1)
    p1 = SpatialDropout2D(dropout)(p1)

    c2 = conv2d_block(p1, n_filters * 2, kernel_size = 3, batchnorm =
batchnorm)
    p2 = MaxPooling2D((2, 2))(c2)
    p2 = SpatialDropout2D(dropout)(p2)

    c3 = conv2d_block(p2, n_filters * 4, kernel_size = 3, batchnorm =
batchnorm)
    p3 = MaxPooling2D((2, 2))(c3)
    p3 = SpatialDropout2D(dropout)(p3)

    c4 = conv2d_block(p3, n_filters * 8, kernel_size = 3, batchnorm =
batchnorm)
    p4 = MaxPooling2D((2, 2))(c4)
    p4 = SpatialDropout2D(dropout)(p4)

    c5 = conv2d_block(p4, n_filters = n_filters * 16, kernel_size = 3,
batchnorm = batchnorm)

    # Expansive Path
    u6 = Conv2DTranspose(n_filters * 8, (3, 3), strides = (2, 2),
padding = 'same')(c5)
    u6 = concatenate([u6, c4])
    u6 = SpatialDropout2D(dropout)(u6)
    c6 = conv2d_block(u6, n_filters * 8, kernel_size = 3, batchnorm =
```

```
batchnorm)

    u7 = Conv2DTranspose(n_filters * 4, (3, 3), strides = (2, 2),
padding = 'same')(c6)
    u7 = concatenate([u7, c3])
    u7 = SpatialDropout2D(dropout)(u7)
    c7 = conv2d_block(u7, n_filters * 4, kernel_size = 3, batchnorm =
batchnorm)

    u8 = Conv2DTranspose(n_filters * 2, (3, 3), strides = (2, 2),
padding = 'same')(c7)
    u8 = concatenate([u8, c2])
    u8 = SpatialDropout2D(dropout * 2)(u8)
    c8 = conv2d_block(u8, n_filters * 2, kernel_size = 3, batchnorm =
batchnorm)

    u9 = Conv2DTranspose(n_filters * 1, (3, 3), strides = (2, 2),
padding = 'same')(c8)
    u9 = concatenate([u9, c1])
    u9 = SpatialDropout2D(dropout * 2)(u9)
    c9 = conv2d_block(u9, n_filters * 1, kernel_size = 3, batchnorm =
batchnorm)

    outputs = Conv2D(1, (1, 1), activation = 'sigmoid')(c9)
    model = Model(inputs=[input_img], outputs=[outputs])
    return model

from tensorflow.keras.layers import Input, Conv2D, BatchNormalization,
Activation, MaxPooling2D, Dropout, Conv2DTranspose, concatenate
from tensorflow.keras.models import Model
input_img = Input(shape=(512,512,3))
model = get_unet(input_img, n_filters = 32, dropout = 0.2, batchnorm =
True, n_classes = 1)
#model.summary()
#couldnt get gpu version to work
print("TF version:", tf.__version__)
print("Physical GPUs:", tf.config.list_physical_devices('GPU'))
print("Logical GPUs:", tf.config.list_logical_devices('GPU'))
print("GPU device name:", tf.test.gpu_device_name())
```

```
TF version: 2.19.0
Physical GPUs: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU')]
Logical GPUs: [LogicalDevice(name='/device:GPU:0', device_type='GPU')]
GPU device name: /device:GPU:0
```

Task 1.2: Train the model, and plot the training history

Feel free to use the `plot_training_history` function from the provided library `utilities.py`.

```
from tensorflow.keras.metrics import FalseNegatives, FalsePositives,
TrueNegatives, TruePositives
from utilities import f1_score, plot_training_history
from tensorflow.keras.losses import Dice

#for more reproducibility
tf.random.set_seed(42)
np.random.seed(42)

optimizer = ks.optimizers.Adam(learning_rate=1e-4)

def dice_bce_loss(y_true, y_pred):
  bce = tf.keras.losses.BinaryCrossentropy()
  dice = tf.keras.losses.Dice()
  bce_loss = bce(y_true, y_pred)
  dice_loss = dice(y_true, y_pred)
  return 0.9 * bce_loss + 0.1 * dice_loss


model.compile(
    optimizer=optimizer,
    loss=dice_bce_loss,
    metrics=[
        FalseNegatives(),
        FalsePositives(),
        TrueNegatives(),
        TruePositives(),
        f1_score,
        tf.keras.metrics.MeanIoU(num_classes=2)
    ]
)

training_history = model.fit(train_aug_ds, validation_data=val_ds,
epochs=20)
plot_training_history(training_history)
```

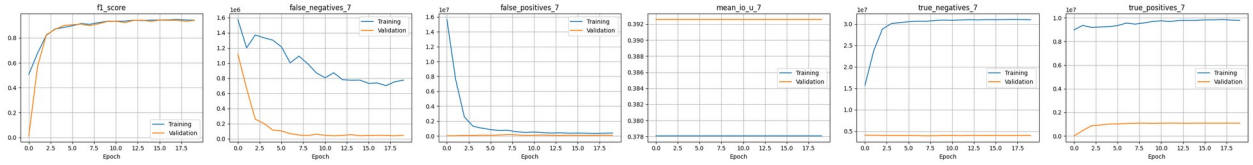
```
Epoch 1/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 79s 1s/step - f1_score: 0.4522 -
false_negatives_7: 1027943.3750 - false_positives_7: 8956829.0000 -
loss: 0.7230 - mean_io_u_7: 0.3741 - true_negatives_7: 7428244.5000 -
true_positives_7: 4581504.5000 - val_f1_score: 0.0123 -
val_false_negatives_7: 1119571.0000 - val_false_positives_7:
19038.0000 - val_loss: 0.4709 - val_mean_io_u_7: 0.3926 -
val_true_negatives_7: 4097297.0000 - val_true_positives_7: 6974.0000
Epoch 2/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 72s 1s/step - f1_score: 0.6433 -
false_negatives_7: 584817.0625 - false_positives_7: 4520124.0000 -
loss: 0.5452 - mean_io_u_7: 0.3789 - true_negatives_7: 11991154.0000 -
true_positives_7: 4898426.0000 - val_f1_score: 0.5683 -
val_false_negatives_7: 664246.0000 - val_false_positives_7: 30451.0000
```

```
- val_loss: 0.3587 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4085884.0000 - val_true_positives_7: 462299.0000
Epoch 3/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 73s 1s/step - f1_score: 0.8017 -
false_negatives_7: 709709.1875 - false_positives_7: 1545642.1250 -
loss: 0.3783 - mean_io_u_7: 0.3783 - true_negatives_7: 14928206.0000 -
true_positives_7: 4810964.0000 - val_f1_score: 0.8251 -
val_false_negatives_7: 258223.0000 - val_false_positives_7: 90440.0000
- val_loss: 0.2864 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4025895.0000 - val_true_positives_7: 868322.0000
Epoch 4/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 71s 1s/step - f1_score: 0.8608 -
false_negatives_7: 730200.0000 - false_positives_7: 731851.3125 -
loss: 0.3057 - mean_io_u_7: 0.3752 - true_negatives_7: 15680689.0000 -
true_positives_7: 4851779.5000 - val_f1_score: 0.8679 -
val_false_negatives_7: 201873.0000 - val_false_positives_7: 75290.0000
- val_loss: 0.2624 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4041045.0000 - val_true_positives_7: 924672.0000
Epoch 5/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 72s 1s/step - f1_score: 0.8735 -
false_negatives_7: 712178.0000 - false_positives_7: 566461.9375 -
loss: 0.2802 - mean_io_u_7: 0.3783 - true_negatives_7: 15936888.0000 -
true_positives_7: 4778993.0000 - val_f1_score: 0.8994 -
val_false_negatives_7: 114265.0000 - val_false_positives_7:
117491.0000 - val_loss: 0.2551 - val_mean_io_u_7: 0.3926 -
val_true_negatives_7: 3998844.0000 - val_true_positives_7:
1012280.0000
Epoch 6/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 71s 1s/step - f1_score: 0.8961 -
false_negatives_7: 647428.6250 - false_positives_7: 475347.6250 -
loss: 0.2425 - mean_io_u_7: 0.3759 - true_negatives_7: 15937142.0000 -
true_positives_7: 4934603.0000 - val_f1_score: 0.9062 -
val_false_negatives_7: 102950.0000 - val_false_positives_7:
105293.0000 - val_loss: 0.2292 - val_mean_io_u_7: 0.3926 -
val_true_negatives_7: 4011042.0000 - val_true_positives_7:
1023595.0000
Epoch 7/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 74s 1s/step - f1_score: 0.9074 -
false_negatives_7: 520653.8125 - false_positives_7: 405411.6875 -
loss: 0.2254 - mean_io_u_7: 0.3827 - true_negatives_7: 16111183.0000 -
true_positives_7: 4957271.0000 - val_f1_score: 0.9105 -
val_false_negatives_7: 68154.0000 - val_false_positives_7: 137134.0000
- val_loss: 0.2252 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
3979201.0000 - val_true_positives_7: 1058391.0000
Epoch 8/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 72s 1s/step - f1_score: 0.9052 -
false_negatives_7: 624629.3125 - false_positives_7: 396247.6250 -
loss: 0.2174 - mean_io_u_7: 0.3758 - true_negatives_7: 15977425.0000 -
true_positives_7: 4996219.5000 - val_f1_score: 0.8982 -
```

```
val_false_negatives_7: 49760.0000 - val_false_positives_7: 191807.0000
- val_loss: 0.2445 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
3924528.0000 - val_true_positives_7: 1076785.0000
Epoch 9/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 73s 1s/step - f1_score: 0.9217 -
false_negatives_7: 521089.0938 - false_positives_7: 290441.4062 -
loss: 0.1938 - mean_io_u_7: 0.3796 - true_negatives_7: 16211869.0000 -
true_positives_7: 4971120.0000 - val_f1_score: 0.9136 -
val_false_negatives_7: 42939.0000 - val_false_positives_7: 158552.0000
- val_loss: 0.2116 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
3957783.0000 - val_true_positives_7: 1083606.0000
Epoch 10/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 72s 1s/step - f1_score: 0.9327 -
false_negatives_7: 451870.5000 - false_positives_7: 260637.6875 -
loss: 0.1794 - mean_io_u_7: 0.3799 - true_negatives_7: 16254564.0000 -
true_positives_7: 5027450.0000 - val_f1_score: 0.9319 -
val_false_negatives_7: 59145.0000 - val_false_positives_7: 95425.0000
- val_loss: 0.1765 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4020910.0000 - val_true_positives_7: 1067400.0000
Epoch 11/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 74s 1s/step - f1_score: 0.9372 -
false_negatives_7: 413118.7500 - false_positives_7: 261187.7344 -
loss: 0.1671 - mean_io_u_7: 0.3786 - true_negatives_7: 16216171.0000 -
true_positives_7: 5104043.0000 - val_f1_score: 0.9342 -
val_false_negatives_7: 46154.0000 - val_false_positives_7: 108568.0000
- val_loss: 0.1651 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4007767.0000 - val_true_positives_7: 1080391.0000
Epoch 12/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 71s 1s/step - f1_score: 0.9320 -
false_negatives_7: 500537.1250 - false_positives_7: 242058.4375 -
loss: 0.1672 - mean_io_u_7: 0.3745 - true_negatives_7: 16107936.0000 -
true_positives_7: 5143989.0000 - val_f1_score: 0.9236 -
val_false_negatives_7: 37763.0000 - val_false_positives_7: 144221.0000
- val_loss: 0.1882 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
3972114.0000 - val_true_positives_7: 1088782.0000
Epoch 13/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 73s 1s/step - f1_score: 0.9368 -
false_negatives_7: 423114.3125 - false_positives_7: 214686.1406 -
loss: 0.1592 - mean_io_u_7: 0.3778 - true_negatives_7: 16271527.0000 -
true_positives_7: 5085193.0000 - val_f1_score: 0.9405 -
val_false_negatives_7: 44789.0000 - val_false_positives_7: 91746.0000
- val_loss: 0.1493 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4024589.0000 - val_true_positives_7: 1081756.0000
Epoch 14/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 72s 1s/step - f1_score: 0.9378 -
false_negatives_7: 418171.0625 - false_positives_7: 234128.1719 -
loss: 0.1513 - mean_io_u_7: 0.3756 - true_negatives_7: 16162600.0000 -
true_positives_7: 5179621.0000 - val_f1_score: 0.9425 -
val_false_negatives_7: 54269.0000 - val_false_positives_7: 78601.0000
```

```
- val_loss: 0.1470 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4037734.0000 - val_true_positives_7: 1072276.0000
Epoch 15/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 74s 1s/step - f1_score: 0.9437 -
false_negatives_7: 398994.5000 - false_positives_7: 199090.8281 -
loss: 0.1417 - mean_io_u_7: 0.3780 - true_negatives_7: 16240809.0000 -
true_positives_7: 5155625.5000 - val_f1_score: 0.9337 -
val_false_negatives_7: 39652.0000 - val_false_positives_7: 111409.0000
- val_loss: 0.1501 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4004926.0000 - val_true_positives_7: 1086893.0000
Epoch 16/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 72s 1s/step - f1_score: 0.9454 -
false_negatives_7: 373536.5000 - false_positives_7: 214018.4844 -
loss: 0.1390 - mean_io_u_7: 0.3794 - true_negatives_7: 16230383.0000 -
true_positives_7: 5176583.5000 - val_f1_score: 0.9426 -
val_false_negatives_7: 43038.0000 - val_false_positives_7: 89886.0000
- val_loss: 0.1318 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4026449.0000 - val_true_positives_7: 1083507.0000
Epoch 17/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 73s 1s/step - f1_score: 0.9447 -
false_negatives_7: 390575.2500 - false_positives_7: 199212.8594 -
loss: 0.1334 - mean_io_u_7: 0.3790 - true_negatives_7: 16225619.0000 -
true_positives_7: 5179114.0000 - val_f1_score: 0.9411 -
val_false_negatives_7: 44736.0000 - val_false_positives_7: 90271.0000
- val_loss: 0.1273 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4026064.0000 - val_true_positives_7: 1081809.0000
Epoch 18/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 71s 1s/step - f1_score: 0.9431 -
false_negatives_7: 383682.4375 - false_positives_7: 186337.2500 -
loss: 0.1310 - mean_io_u_7: 0.3760 - true_negatives_7: 16225337.0000 -
true_positives_7: 5199164.5000 - val_f1_score: 0.9417 -
val_false_negatives_7: 44245.0000 - val_false_positives_7: 85629.0000
- val_loss: 0.1342 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4030706.0000 - val_true_positives_7: 1082300.0000
Epoch 19/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 73s 1s/step - f1_score: 0.9421 -
false_negatives_7: 421335.6250 - false_positives_7: 192010.3438 -
loss: 0.1315 - mean_io_u_7: 0.3779 - true_negatives_7: 16262077.0000 -
true_positives_7: 5119097.5000 - val_f1_score: 0.9339 -
val_false_negatives_7: 38331.0000 - val_false_positives_7: 121287.0000
- val_loss: 0.1424 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
3995048.0000 - val_true_positives_7: 1088214.0000
Epoch 20/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 73s 1s/step - f1_score: 0.9375 -
false_negatives_7: 423696.8125 - false_positives_7: 209469.6406 -
loss: 0.1337 - mean_io_u_7: 0.3812 - true_negatives_7: 16292898.0000 -
true_positives_7: 5068457.0000 - val_f1_score: 0.9406 -
val_false_negatives_7: 47458.0000 - val_false_positives_7: 84804.0000
```

```
- val_loss: 0.1205 - val_mean_io_u_7: 0.3926 - val_true_negatives_7:
4031531.0000 - val_true_positives_7: 1079087.0000
```



true negatives and positives having relatively small changes, while false negatives and false positives both decreasing during training suggests that the areas at the edge of the lungs are hardest to map correctly, and the lungs and background themselves are easier.

```
predictions = model.predict(test_ds, verbose=1)

f1 = f1_score(y_true=y_true, y_pred=predictions)
print(f1)

5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 163ms/step
tf.Tensor(0.69339305, shape=(), dtype=float32)
```

Task 1.3: Visualize model predictions

Make a plot that illustrates the original image, the predicted mask, and the ground truth mask. You can use visualize_predictions() in `utilities.py`

```
import matplotlib.pyplot as plt

for imgs, masks in test_ds.take(1):
    preds = model.predict(imgs)
    preds_bin = (preds > 0.5).astype(int)

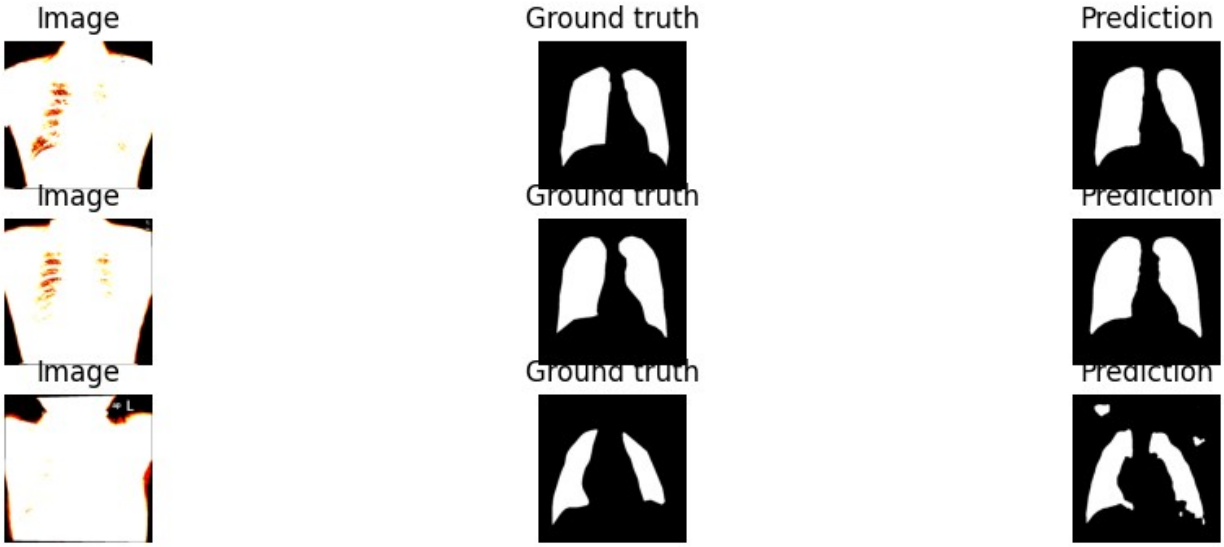
    plt.figure(figsize=(12, 4))
    for i in range(min(3, imgs.shape[0])):
        plt.subplot(3, 3, i*3+1)
        plt.imshow(imgs[i])
        plt.title("Image")
        plt.axis("off")

        plt.subplot(3, 3, i*3+2)
        plt.imshow(masks[i,:,:,0], cmap="gray")
        plt.title("Ground truth")
        plt.axis("off")

        plt.subplot(3, 3, i*3+3)
        plt.imshow(preds_bin[i,:,:,0], cmap="gray")
        plt.title("Prediction")
        plt.axis("off")
    plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 262ms/step

WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-123.68..151.061].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-123.68..151.061].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-123.68..151.061].
```



Task 2: Implementing U-Net with Transfer Learning

Implement a model with the U-net structure that you have learned about in the lectures, but now with a pre-trained backbone. There are many pre-trained back-bones to choose from. Pick freely from the selection here tf.keras.applications, or here Keras model scores (nicer table in the second link). Feel free to experiment with the number of layers, loss-function, batch-normalization, etc. Many of the backbones available are quite big, so you might find it quite time-consuming to train them on your personal computers.

For those with a dedicated graphics card (NVIDIA and AMD) Tensorflow or PyTorch (not syllabus)

And wants to experiment with their own compute resources (can be a lot of fun)

Tensorflow: https://learn.microsoft.com/en-us/windows/ai/directml/gpu-tensorflow-plugin

PyTorch: https://learn.microsoft.com/en-us/windows/ai/directml/pytorch-windows

Task 2.1: Transfer Learning Model Implementation

In this task, we will implement a U-Net model that uses the pre-trained weights of popular public networks (e.g., **VGG16** and **ResNet50**) as encoders. This is a common transfer learning strategy for segmentation.

Example 1: U-Net with VGG16 backbone

from tensorflow.keras.applications import VGG16

from tensorflow.keras.applications import ResNet50

```
#editing the lecture notes for unet to include a backbone with
ResNet50
from tensorflow.keras.layers import UpSampling2D
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications import ResNet50

from tensorflow.keras.layers import SpatialDropout2D

def conv2d_block(input_tensor, n_filters, kernel_size = 3, batchnorm =
True):
    """Function to add 2 convolutional layers with the parameters
passed to it"""
    # first layer
    x = Conv2D(filters = n_filters, kernel_size = (kernel_size,
kernel_size),\
               kernel_initializer = 'he_normal', padding = 'same')
(input_tensor)
    if batchnorm:
        x = BatchNormalization()(x)
    x = Activation('relu')(x)

    # second layer
    x = Conv2D(filters = n_filters, kernel_size = (kernel_size,
kernel_size),\
               kernel_initializer = 'he_normal', padding = 'same')(x)
    if batchnorm:
        x = BatchNormalization()(x)
    x = Activation('relu')(x)

    return x


def get_unet_with_resnet50(input_shape=(512, 512, 3), dropout = 0.1,
batchnorm = True, n_classes = 2):

    input_img = Input(shape=input_shape)
    #ResNet50 as backbone
    base_model = ResNet50(weights='imagenet', include_top=False,
```

```
input_tensor=input_img)

    #encoder feature maps
    c1 = base_model.get_layer('conv1_relu').output        # /2
    c2 = base_model.get_layer('conv2_block3_out').output  # /4
    c3 = base_model.get_layer('conv3_block4_out').output  # /8
    c4 = base_model.get_layer('conv4_block6_out').output  # /16
    c5 = base_model.get_layer('conv5_block3_out').output  # /32

    # Expansive Path
    u6 = Conv2DTranspose(512, (3, 3), strides = (2, 2), padding =
'same')(c5)
    u6 = concatenate([u6, c4])
    u6 = SpatialDropout2D(dropout)(u6)
    c6 = conv2d_block(u6, 512, kernel_size = 3, batchnorm = batchnorm)

    u7 = Conv2DTranspose(256, (3, 3), strides = (2, 2), padding =
'same')(c6)
    u7 = concatenate([u7, c3])
    u7 = SpatialDropout2D(dropout)(u7)
    c7 = conv2d_block(u7, 256, kernel_size = 3, batchnorm = batchnorm)

    u8 = Conv2DTranspose(128, (3, 3), strides = (2, 2), padding =
'same')(c7)
    u8 = concatenate([u8, c2])
    u8 = SpatialDropout2D(dropout * 2)(u8)
    c8 = conv2d_block(u8, 128, kernel_size = 3, batchnorm = batchnorm)

    u9 = Conv2DTranspose(64, (3, 3), strides = (2, 2), padding =
'same')(c8)
    u9 = concatenate([u9, c1])
    u9 = SpatialDropout2D(dropout * 2)(u9)
    c9 = conv2d_block(u9, 64, kernel_size = 3, batchnorm = batchnorm)

    u10 = UpSampling2D(size=(2,2))(c9)

    outputs = Conv2D(n_classes, (1, 1), activation = 'sigmoid')(u10)
    model = Model(inputs=input_img, outputs=outputs)
    return model
```

Task 2.2: Train the transfer learning model and plot the training history

Feel free to use the plot_training_history function from the provided library utilities.py

```
from utilities import f1_score
optimizer = ks.optimizers.Adam(learning_rate=1e-4)

model_resnet50 = get_unet_with_resnet50(
```

```
    input_shape=(512, 512, 3),
    dropout = 0.2,
    batchnorm = True,
    n_classes = 1
)

model_resnet50.compile(
    optimizer=optimizer,
    loss="binary_crossentropy",
    metrics=[
        tf.keras.metrics.FalseNegatives(),
        tf.keras.metrics.FalsePositives(),
        tf.keras.metrics.TrueNegatives(),
        tf.keras.metrics.TruePositives(),
        f1_score,
        tf.keras.metrics.MeanIoU(num_classes=2)
    ]
)

training_history = model_resnet50.fit(
    train_aug_ds,
    validation_data = val_ds,
    epochs=20
    )

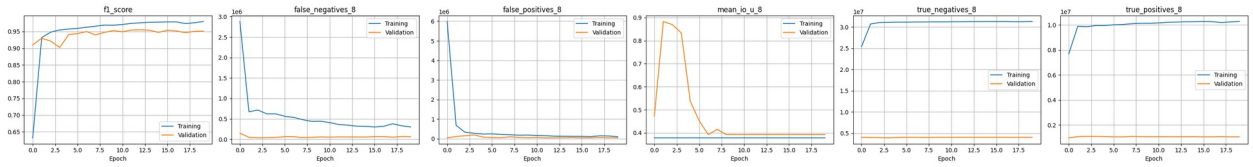
plot_training_history(training_history)


Epoch 1/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 117s 2s/step - f1_score: 0.4906 -
false_negatives_8: 2009478.8750 - false_positives_8: 3774534.7500 -
loss: 0.5609 - mean_io_u_8: 0.3804 - true_negatives_8: 12673671.0000 -
true_positives_8: 3536836.2500 - val_f1_score: 0.9101 -
val_false_negatives_8: 148799.0000 - val_false_positives_8: 45938.0000
- val_loss: 0.1546 - val_mean_io_u_8: 0.4728 - val_true_negatives_8:
4070397.0000 - val_true_positives_8: 977746.0000
Epoch 2/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 115s 2s/step - f1_score: 0.9209 -
false_negatives_8: 346782.4688 - false_positives_8: 429588.4688 -
loss: 0.2005 - mean_io_u_8: 0.3757 - true_negatives_8: 15934573.0000 -
true_positives_8: 5283577.0000 - val_f1_score: 0.9298 -
val_false_negatives_8: 46654.0000 - val_false_positives_8: 114189.0000
- val_loss: 0.1831 - val_mean_io_u_8: 0.8831 - val_true_negatives_8:
4002146.0000 - val_true_positives_8: 1079891.0000
Epoch 3/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 115s 2s/step - f1_score: 0.9484 -
false_negatives_8: 350305.0000 - false_positives_8: 176798.7344 -
loss: 0.1105 - mean_io_u_8: 0.3801 - true_negatives_8: 16360631.0000 -
true_positives_8: 5106786.0000 - val_f1_score: 0.9214 -
val_false_negatives_8: 33141.0000 - val_false_positives_8: 151696.0000
```

```
- val_loss: 0.2020 - val_mean_io_u_8: 0.8699 - val_true_negatives_8:
3964639.0000 - val_true_positives_8: 1093404.0000
Epoch 4/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 115s 2s/step - f1_score: 0.9525 -
false_negatives_8: 327724.5000 - false_positives_8: 147470.6562 -
loss: 0.0908 - mean_io_u_8: 0.3792 - true_negatives_8: 16357522.0000 -
true_positives_8: 5161804.0000 - val_f1_score: 0.9023 -
val_false_negatives_8: 36494.0000 - val_false_positives_8: 196665.0000
- val_loss: 0.2684 - val_mean_io_u_8: 0.8335 - val_true_negatives_8:
3919670.0000 - val_true_positives_8: 1090051.0000
Epoch 5/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - f1_score: 0.9558 -
false_negatives_8: 335768.9688 - false_positives_8: 122109.6562 -
loss: 0.0808 - mean_io_u_8: 0.3796 - true_negatives_8: 16352129.0000 -
true_positives_8: 5184513.5000 - val_f1_score: 0.9406 -
val_false_negatives_8: 46151.0000 - val_false_positives_8: 86386.0000
- val_loss: 0.1130 - val_mean_io_u_8: 0.5367 - val_true_negatives_8:
4029949.0000 - val_true_positives_8: 1080394.0000
Epoch 6/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 115s 2s/step - f1_score: 0.9615 -
false_negatives_8: 288032.7500 - false_positives_8: 123594.6562 -
loss: 0.0706 - mean_io_u_8: 0.3791 - true_negatives_8: 16387068.0000 -
true_positives_8: 5195826.0000 - val_f1_score: 0.9440 -
val_false_negatives_8: 62820.0000 - val_false_positives_8: 56003.0000
- val_loss: 0.0969 - val_mean_io_u_8: 0.4519 - val_true_negatives_8:
4060332.0000 - val_true_positives_8: 1063725.0000
Epoch 7/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 114s 2s/step - f1_score: 0.9617 -
false_negatives_8: 290505.0000 - false_positives_8: 107886.4844 -
loss: 0.0679 - mean_io_u_8: 0.3779 - true_negatives_8: 16333943.0000 -
true_positives_8: 5262188.5000 - val_f1_score: 0.9499 -
val_false_negatives_8: 63198.0000 - val_false_positives_8: 49517.0000
- val_loss: 0.0857 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4066818.0000 - val_true_positives_8: 1063347.0000
Epoch 8/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - f1_score: 0.9662 -
false_negatives_8: 253904.2969 - false_positives_8: 103014.1953 -
loss: 0.0605 - mean_io_u_8: 0.3799 - true_negatives_8: 16353240.0000 -
true_positives_8: 5284363.0000 - val_f1_score: 0.9398 -
val_false_negatives_8: 39876.0000 - val_false_positives_8: 101027.0000
- val_loss: 0.1181 - val_mean_io_u_8: 0.4152 - val_true_negatives_8:
4015308.0000 - val_true_positives_8: 1086669.0000
Epoch 9/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 117s 2s/step - f1_score: 0.9700 -
false_negatives_8: 224686.1719 - false_positives_8: 91006.6328 - loss:
0.0542 - mean_io_u_8: 0.3792 - true_negatives_8: 16400078.0000 -
true_positives_8: 5278751.5000 - val_f1_score: 0.9476 -
val_false_negatives_8: 49047.0000 - val_false_positives_8: 69882.0000
- val_loss: 0.0911 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
```

```
4046453.0000 - val_true_positives_8: 1077498.0000
Epoch 10/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 117s 2s/step - f1_score: 0.9712 -
false_negatives_8: 217986.4688 - false_positives_8: 88756.2891 - loss:
0.0526 - mean_io_u_8: 0.3790 - true_negatives_8: 16422607.0000 -
true_positives_8: 5265171.0000 - val_f1_score: 0.9527 -
val_false_negatives_8: 57666.0000 - val_false_positives_8: 48989.0000
- val_loss: 0.0765 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4067346.0000 - val_true_positives_8: 1068879.0000
Epoch 11/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - f1_score: 0.9709 -
false_negatives_8: 221834.9531 - false_positives_8: 87017.7812 - loss:
0.0510 - mean_io_u_8: 0.3795 - true_negatives_8: 16397053.0000 -
true_positives_8: 5288614.0000 - val_f1_score: 0.9492 -
val_false_negatives_8: 52502.0000 - val_false_positives_8: 61085.0000
- val_loss: 0.0797 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4055250.0000 - val_true_positives_8: 1074043.0000
Epoch 12/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - f1_score: 0.9728 -
false_negatives_8: 198145.7031 - false_positives_8: 79535.4141 - loss:
0.0479 - mean_io_u_8: 0.3771 - true_negatives_8: 16371585.0000 -
true_positives_8: 5345254.5000 - val_f1_score: 0.9544 -
val_false_negatives_8: 57497.0000 - val_false_positives_8: 42660.0000
- val_loss: 0.0752 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4073675.0000 - val_true_positives_8: 1069048.0000
Epoch 13/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - f1_score: 0.9754 -
false_negatives_8: 181703.6406 - false_positives_8: 69099.6094 - loss:
0.0433 - mean_io_u_8: 0.3808 - true_negatives_8: 16432372.0000 -
true_positives_8: 5311346.0000 - val_f1_score: 0.9554 -
val_false_negatives_8: 57085.0000 - val_false_positives_8: 43728.0000
- val_loss: 0.0743 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4072607.0000 - val_true_positives_8: 1069460.0000
Epoch 14/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - f1_score: 0.9783 -
false_negatives_8: 163135.3125 - false_positives_8: 65488.0234 - loss:
0.0386 - mean_io_u_8: 0.3770 - true_negatives_8: 16345364.0000 -
true_positives_8: 5420532.5000 - val_f1_score: 0.9537 -
val_false_negatives_8: 55908.0000 - val_false_positives_8: 47519.0000
- val_loss: 0.0762 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4068816.0000 - val_true_positives_8: 1070637.0000
Epoch 15/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - f1_score: 0.9781 -
false_negatives_8: 160650.0000 - false_positives_8: 64968.0000 - loss:
0.0379 - mean_io_u_8: 0.3776 - true_negatives_8: 16375318.0000 -
true_positives_8: 5393584.0000 - val_f1_score: 0.9476 -
val_false_negatives_8: 55652.0000 - val_false_positives_8: 57387.0000
- val_loss: 0.0808 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4058948.0000 - val_true_positives_8: 1070893.0000
```

```
Epoch 16/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - f1_score: 0.9792 -
false_negatives_8: 152412.6875 - false_positives_8: 62402.2695 - loss:
0.0358 - mean_io_u_8: 0.3778 - true_negatives_8: 16372875.0000 -
true_positives_8: 5406832.0000 - val_f1_score: 0.9537 -
val_false_negatives_8: 63154.0000 - val_false_positives_8: 42733.0000
- val_loss: 0.0758 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4073602.0000 - val_true_positives_8: 1063391.0000
Epoch 17/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 115s 2s/step - f1_score: 0.9785 -
false_negatives_8: 165137.9688 - false_positives_8: 57700.0000 - loss:
0.0365 - mean_io_u_8: 0.3784 - true_negatives_8: 16425841.0000 -
true_positives_8: 5345844.0000 - val_f1_score: 0.9516 -
val_false_negatives_8: 62950.0000 - val_false_positives_8: 44213.0000
- val_loss: 0.0728 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4072122.0000 - val_true_positives_8: 1063595.0000
Epoch 18/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 115s 2s/step - f1_score: 0.9759 -
false_negatives_8: 180902.7344 - false_positives_8: 75133.7812 - loss:
0.0390 - mean_io_u_8: 0.3787 - true_negatives_8: 16377078.0000 -
true_positives_8: 5361404.5000 - val_f1_score: 0.9467 -
val_false_negatives_8: 52790.0000 - val_false_positives_8: 63061.0000
- val_loss: 0.0801 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4053274.0000 - val_true_positives_8: 1073755.0000
Epoch 19/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 117s 2s/step - f1_score: 0.9752 -
false_negatives_8: 179051.2188 - false_positives_8: 76862.4375 - loss:
0.0386 - mean_io_u_8: 0.3771 - true_negatives_8: 16322016.0000 -
true_positives_8: 5416591.5000 - val_f1_score: 0.9506 -
val_false_negatives_8: 64166.0000 - val_false_positives_8: 45817.0000
- val_loss: 0.0751 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4070518.0000 - val_true_positives_8: 1062379.0000
Epoch 20/20
40/40 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - f1_score: 0.9794 -
false_negatives_8: 156434.5312 - false_positives_8: 54230.3672 - loss:
0.0331 - mean_io_u_8: 0.3787 - true_negatives_8: 16468293.0000 -
true_positives_8: 5315561.5000 - val_f1_score: 0.9511 -
val_false_negatives_8: 61406.0000 - val_false_positives_8: 47781.0000
- val_loss: 0.0747 - val_mean_io_u_8: 0.3926 - val_true_negatives_8:
4068554.0000 - val_true_positives_8: 1065139.0000
```




```
predictions_resnet50 = model_resnet50.predict(test_ds, verbose=1)
```

```
f1_resnet50 = f1_score(y_true=y_true, y_pred=predictions_resnet50)
print(f1_resnet50)
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 301ms/step
tf.Tensor(0.71691996, shape=(), dtype=float32)
```

Part 2 - Discussion

Question 1: Which model architectures did you explore, and what type of hyperparameter optimization did you try?

Answer 1:

Question 2: Which of the models did you choose to train as your final submission, and how long did it take to train?

Answer 2:

Question 3: What were the biggest challenges with this assignment?

Answer 3:

ANSWER 1

The two models used in this assignment was;

1.The U-net architecture using encoder-decoder structure with given skip connection. The encoders consisted of two convolutional layers with the Relu activation function and the batch normalization. A max pooling layer was added as well for further downsampling.

2- The transfer-learning based U-net with the ResNet50. Here we replaced the U-net encoder with the ResNet50 network. Pretrained layers were frozen during initial training and a decoder was used here to upsample and reconstruct.

We adjusted the filter sizes several times to balance between model capacity and GPU memory use. "Dropout" was also used to reduce overfitting in the assignment, usually between 0.1 and 0.3. This was due to the background in the images which could mess up our final results.

The BCE was used for a loss function. This seemed useful as the total combined loss with the diceloss and a weighted combination maintained the accuracy while improving segmentation overlap.

The learning rate was adjusted during the assignment and led to smoother convergence and less oscillation in the final validation loss.

ANSWER 3

There were several challenges in making this assignment.

Running the assignment with GPU memory was a bit difficult. The images that where 512 x 512 were hard for the GPu memory to work with, and it took quite some time for the code to run. This was adjusted by lowering batch sizes and the number of filters.

Pairing the images with the matching mask was also a bit complicated. There was some trouble for the model to find the "perfect" match, and it took some trial and error to match all of them.

The dice loss function was useful, but it struggled wioth small normalization errors which led to unstable gradients.

The masks themselves were also a challenge. They could be a bit hard to work with considering both resizing and augmentation.